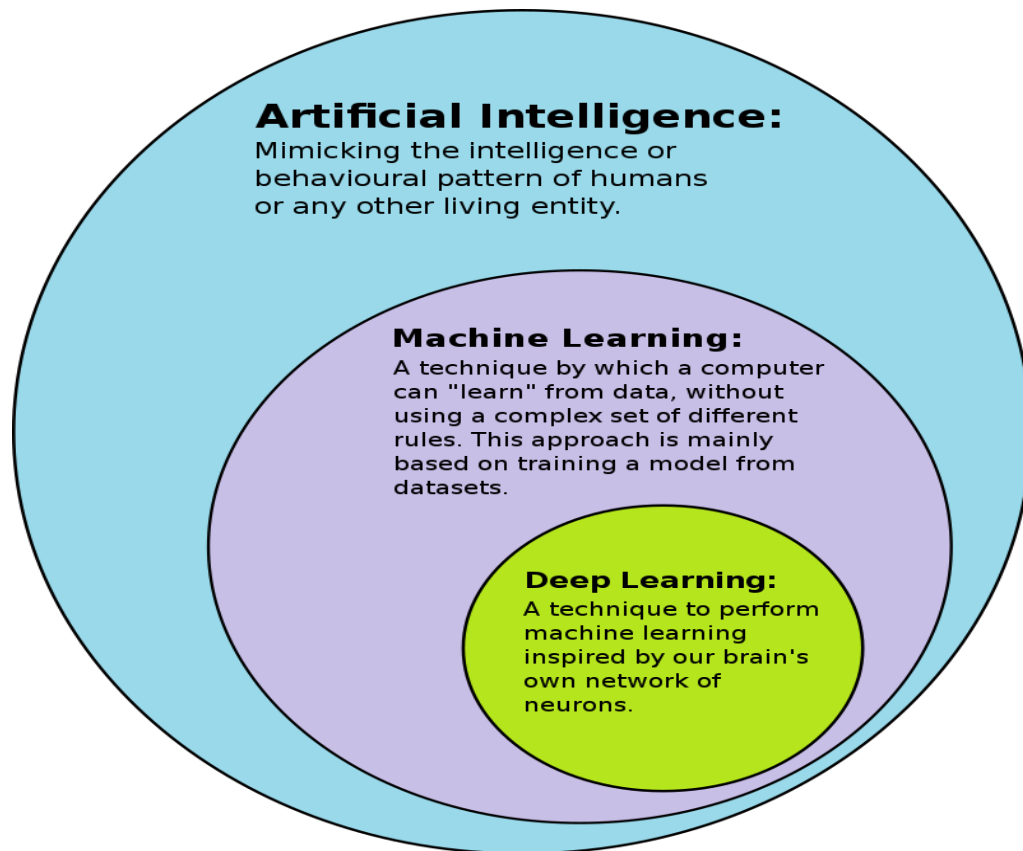


HISTORY OF DEEP LEARNING



- It's one kind of supervised machine learning, in which a computer is provided a training set of examples to learn a function, where each example is a pair of an input and an output from the function.
- The more complex the task, the bigger the training set has to be.
- Ex. Google's voice recognition algorithms
- Responsible for recent advances in computer vision, speech recognition, natural language processing, and audio recognition.

HISTORY OF DEEP LEARNING

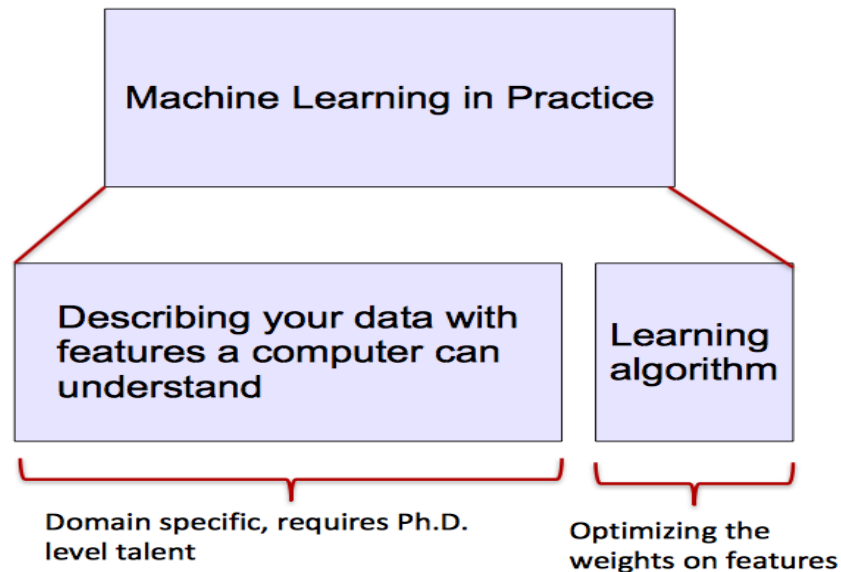
- 1989: Scientists were able to create algorithms that used deep neural networks, but training times for the systems were measured in days, making them impractical for real-world use.
- 1992: Juyang Weng publishes Cresceptron, a method for performing 3-D object recognition automatically from cluttered scenes.
- Mid-2000s: The term “deep learning” begins to gain popularity after a paper by Geoffrey Hinton and Ruslan Salakhutdinov showed how a many-layered neural network could be pre-trained one layer at a time.
- 2009: NIPS Workshop on Deep Learning for Speech Recognition discovers that with a large enough data set, the neural networks don’t need pre-training, and the error rates drop significantly.

HISTORY OF DEEP LEARNING

- 2012: Artificial pattern-recognition algorithms achieve human-level performance on certain tasks. And Google's deep learning algorithm discovers cats.
- 2014: Google buys UK artificial intelligence startup Deepmind for £400m
- 2015: Facebook puts deep learning technology – called DeepFace – into operations to automatically tag and identify Facebook users in photographs. Algorithms perform superior face recognition tasks using deep networks that take into account 120 million parameters.
- 2016: Google DeepMind's algorithm AlphaGo masters the art of the complex board game Go and beats the professional go player Lee Sedol at a highly publicised tournament in Seoul.

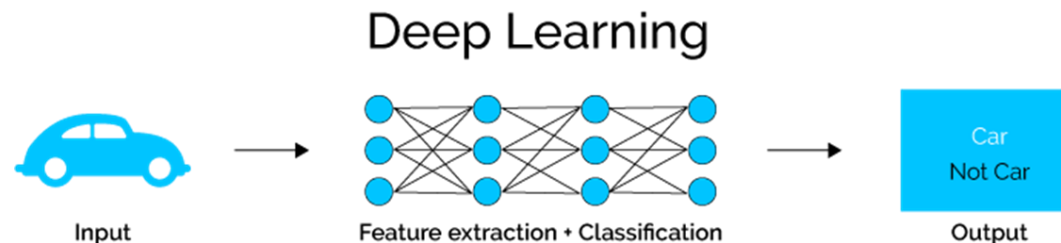
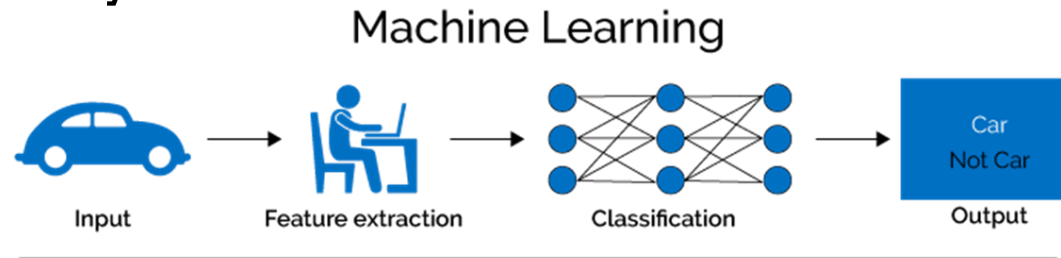
ML VS. DEEP LEARNING

- Most machine learning methods work well because of human-designed representations and input features
- ML becomes just optimizing weights to best make a final prediction



WHAT IS DEEP LEARNING?

- A machine learning subfield of learning representations of data. Exceptional effective at learning patterns.
- Deep learning algorithms attempt to learn (multiple levels of) representation by using a hierarchy of multiple layers
- If you provide the system tons of information, it begins to understand it and respond in useful ways.



WHY IS DL USEFUL?

- Manually designed features are often **over-specified**, **incomplete** and take a **long time to design** and validate
- Learned Features are **easy to adapt**, **fast** to learn
- Deep learning provides a very **flexible**, **universal**, learnable framework for representing world, visual and linguistic information.
- Can learn both unsupervised and supervised
- Utilize large amounts of training data

CHALLENGES IN DL

- Application: Image classification

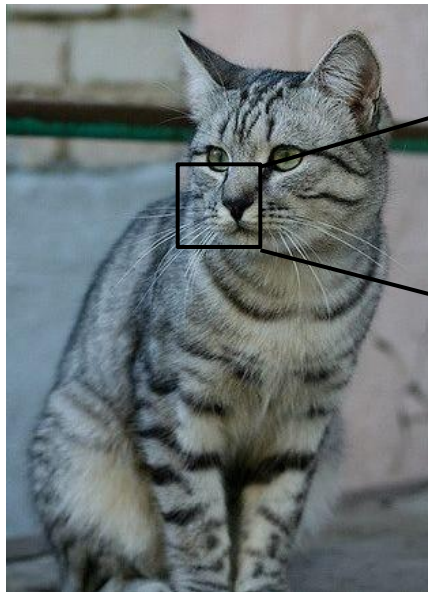


This image by Nikita is
licensed under [CC-BY 2.0](#)

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}

→ cat

THE PROBLEM



This image by Nikita is
licensed under CC-BY 2.0

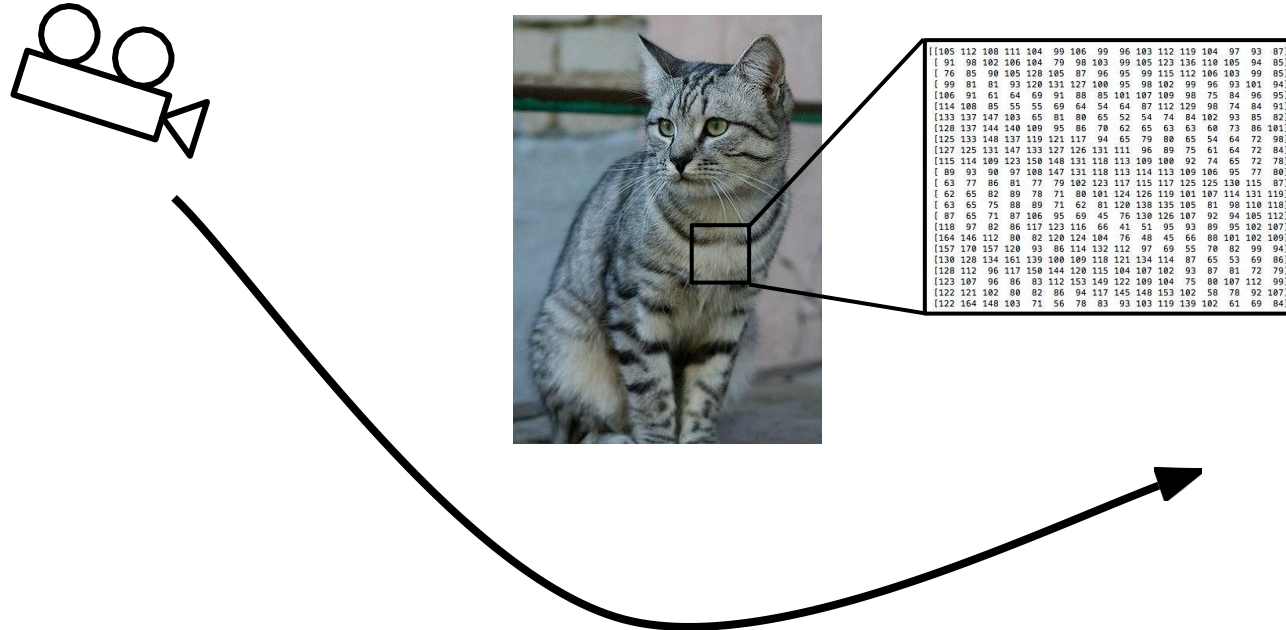
105	112	108	111	104	99	106	99	96	103	112	119	104	97	93	87]
91	98	102	106	104	79	98	103	99	105	123	136	110	105	94	85]
76	85	90	105	128	105	87	96	95	99	115	112	106	103	99	85]
99	81	81	93	120	131	127	100	95	98	102	99	96	93	101	94]
106	91	61	64	69	91	88	85	101	107	109	98	75	84	96	95]
114	108	85	55	55	69	64	54	64	87	112	129	98	74	84	91]
133	137	147	103	65	81	80	65	52	54	74	84	102	93	85	82]
128	137	144	140	109	95	86	70	62	65	63	63	60	73	86	101]
125	133	148	137	119	121	117	94	65	79	80	65	54	64	72	98]
127	125	131	147	133	127	126	131	111	96	89	75	61	64	72	84]
115	114	109	123	150	148	131	118	113	109	100	92	74	65	72	78]
89	93	90	97	108	147	131	118	113	114	113	109	106	95	77	80]
63	77	86	81	77	79	102	123	117	115	117	125	125	130	115	87]
62	65	82	89	78	71	80	101	124	126	119	101	107	114	131	119]
63	65	75	88	89	71	62	81	120	138	135	105	81	98	110	118]
87	65	71	87	106	95	69	45	76	130	126	107	92	94	105	112]
118	97	82	86	117	123	116	66	41	51	95	93	89	95	102	107]
164	146	112	80	82	120	124	104	76	48	45	66	88	101	102	109]
157	170	157	120	93	86	114	132	112	97	69	55	70	82	99	94]
130	128	134	161	139	100	109	118	121	134	114	87	65	53	69	86]
128	112	96	117	150	144	120	115	104	107	102	93	87	81	72	79]
123	107	96	86	83	112	153	149	122	109	104	75	80	107	112	99]
122	121	102	80	82	86	94	117	145	148	153	102	58	78	92	107]
122	107	102	77	67	78	69	69	103	112	99	82	51	69	84]	]

What the computer sees

An image is just a big grid of
numbers between $[0, 255]$:
e.g. $800 \times 600 \times 3$
(3 channels RGB)

CHALLENGES

- Viewpoint orientation: All pixels change when the camera moves



All pixels change when
the camera moves!

CHALLENGES

- Illumination



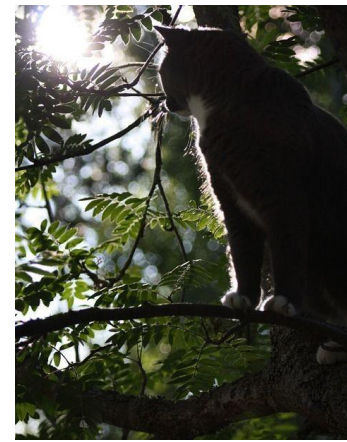
[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

CHALLENGES

- Deformation



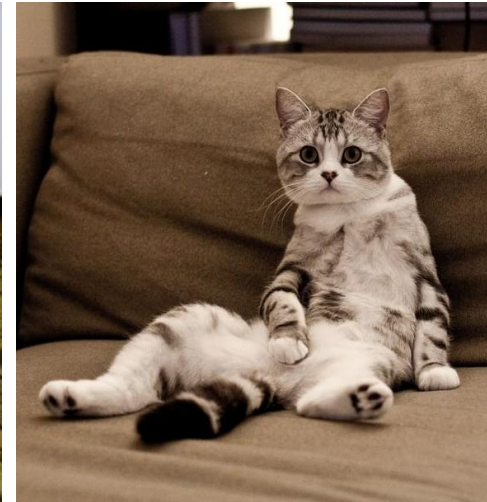
This image by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)



This image by [Umberto Salvagnin](#) is licensed under [CC-BY 2.0](#)



This image by [sare bear](#) is licensed under [CC-BY 2.0](#)



This image by [Tom Thai](#) is licensed under [CC-BY 2.0](#)

CHALLENGES

- Occlusion



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) by [jonsson](#) is licensed under [CC-BY 2.0](#)

CHALLENGES

- Background clutter



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

CHALLENGES

- Intraclass Variation



This image is [CC0 1.0](#) public domain

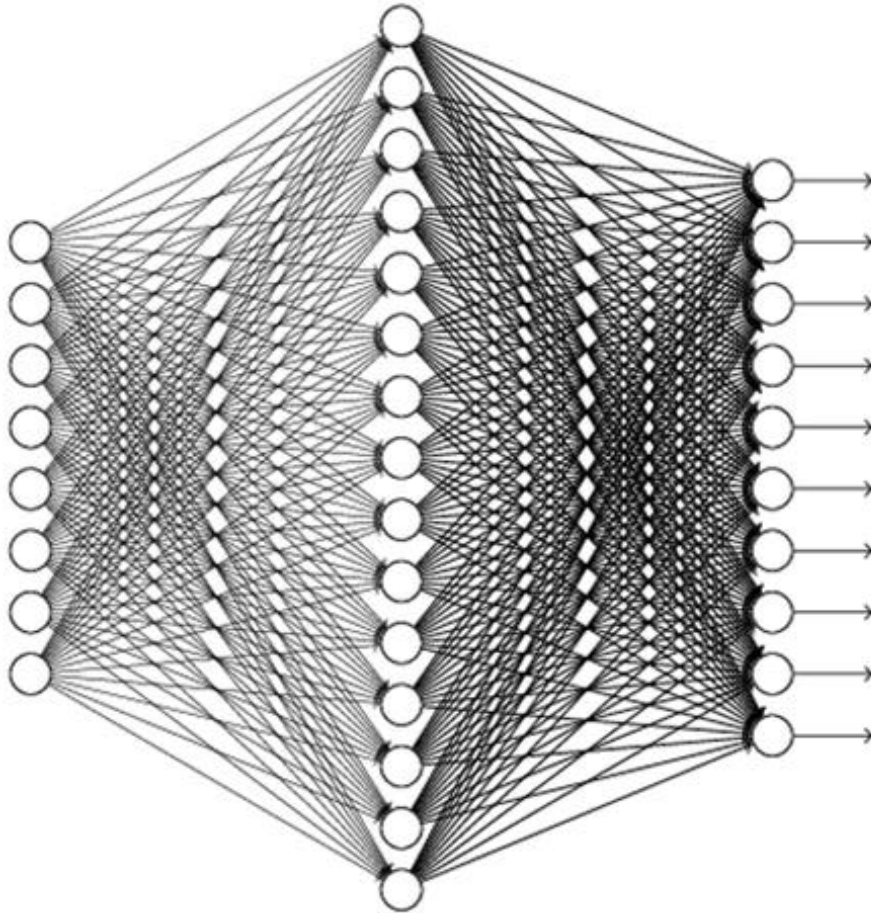
DEEP LEARNING FRAMEWORKS

- It's not practical to implement everything from scratch. Our numpy implementations were to know how NN works.
- There are many good deep learning frameworks.
- Deep learning is now in the phase of doing something with the frameworks and not from scratch to keep on going.
- Here are some of the leading deep learning frameworks:
 - Caffe/ Caffe2
 - CNTK
 - DL4j
 - Keras
 - Lasagne
 - mxnet
 - PaddlePaddle
 - TensorFlow
 - Theano
 - Torch/Pytorch

DEEP LEARNING FRAMEWORKS

- These frameworks are getting better month by month.
- How to choose deep learning framework:
 - Ease of programming (development and deployment)
 - Running speed
 - Truly open (open source with good governance)
- Programming frameworks can not only shorten your coding time but sometimes also perform optimizations that speed up your code.

SHALLOW VS DEEP NETWORK



Shallow Network

- A sufficiently wide neural network with just a single hidden layer can approximate any (reasonable) function given enough training data.
- **Very wide, shallow networks are very good at memorization, but not so good at generalization.**
- If you train the network with every possible input value, a super wide network could eventually memorize the corresponding output value that you want. But that's not useful because for any practical application you won't have every possible input value to train with.

SHALLOW VS DEEP NETWORK

Deep Network

The advantage of multiple layers is that they can learn features at various levels of abstraction.

Example: Image Classification

Layer 1: edges

Layer 2: collections of edges such as shapes

Layer 3: Eyes or noses

.....

Layer n: higher-order features -faces.

Multiple layers are much better at generalizing because they learn all the intermediate features between the raw data and the high-level classification.

MOTIVATION BEHIND CONVNETS

Consider an image of size $200 \times 200 \times 3$ (200 wide, 200 high, 3 color channels)

- a **single fully-connected neuron** in a first hidden layer of a regular Neural Network would have $200 \times 200 \times 3 = 120,000$ weights.

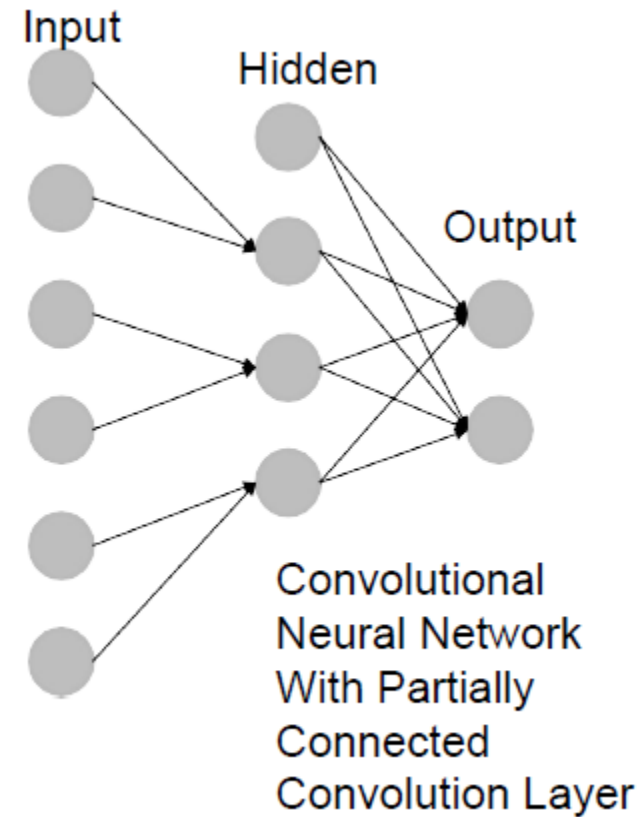
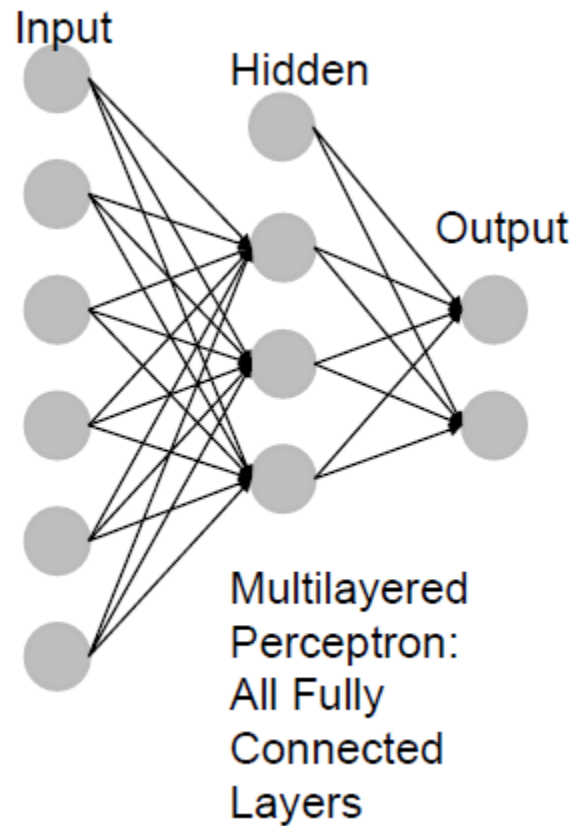
- Due to the presence of several such neurons, this full connectivity is waste and the huge number of parameters would quickly lead to overfitting

- However, in a ConvNet, the neurons in a layer will only be connected to a small region of the layer

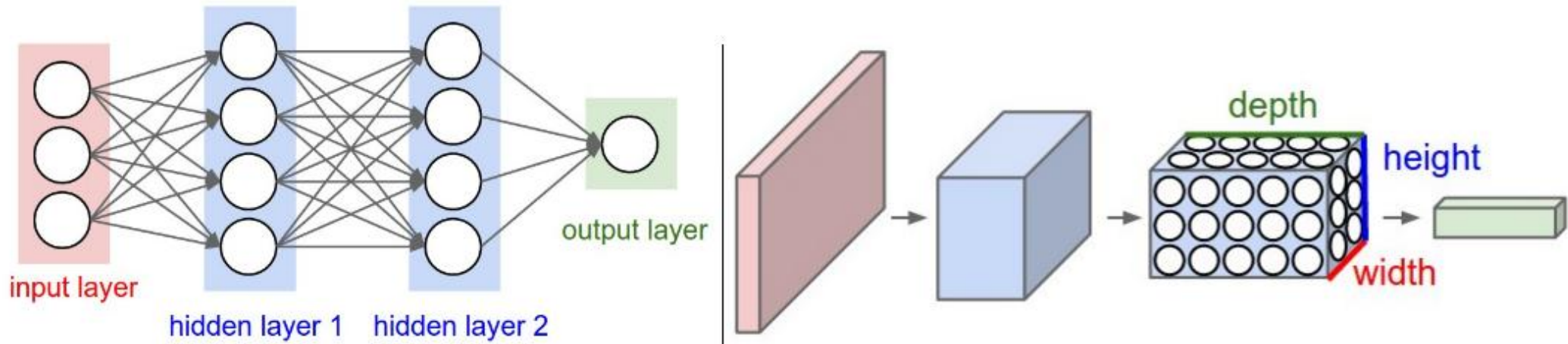
before it, instead of all of the neurons in a fully-connected manner.

- the final output layer would have dimensions $1 \times 1 \times N$, because by the end of the ConvNet architecture we will reduce the full image into a single vector of class scores (for N classes), arranged along the depth dimension

MLP VS CONVNET

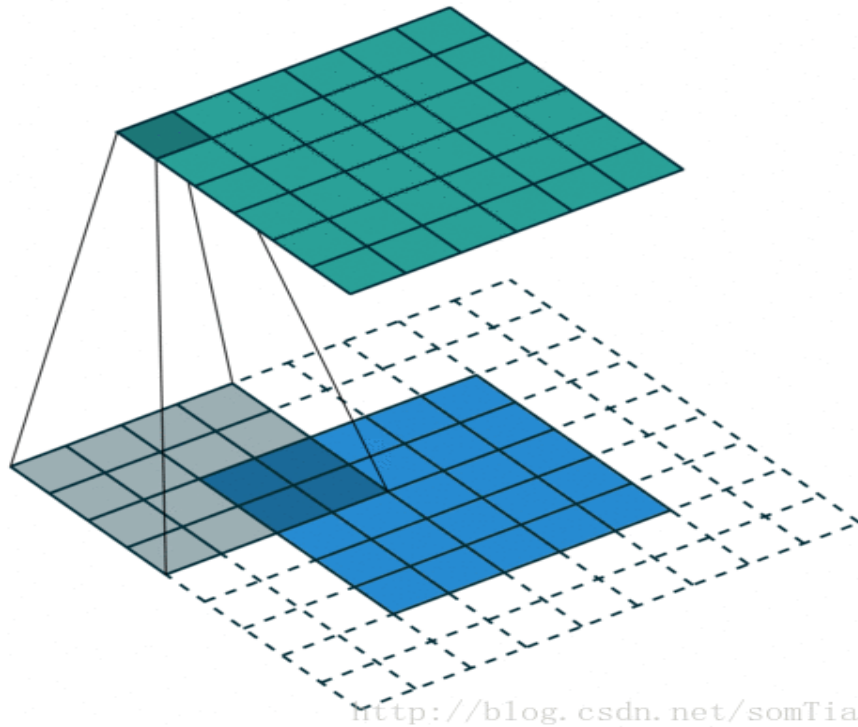


MLP VS CONVNET

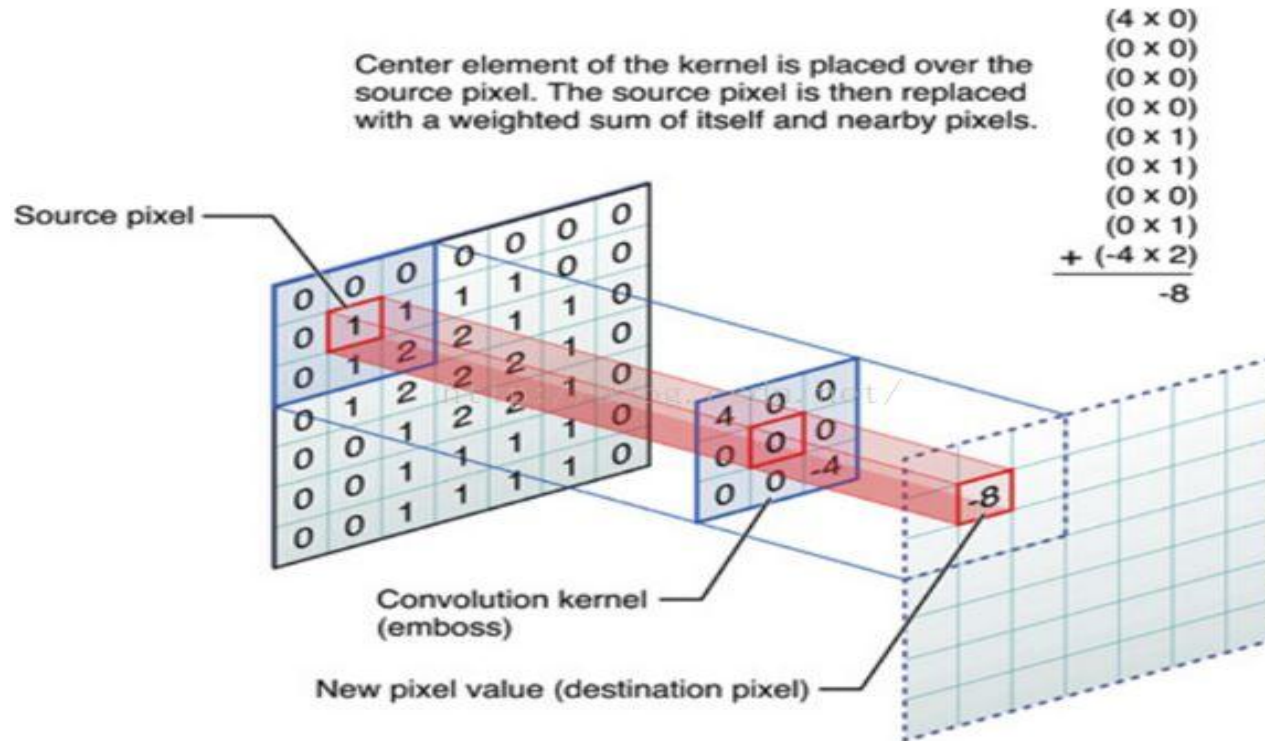


Left: A regular 3-layer Neural Network. Right: A ConvNet arranges its neurons in three dimensions (width, height, depth), as visualized in one of the layers. Every layer of a ConvNet transforms the 3D input volume to a 3D output volume of neuron activations. In this example, the red input layer holds the image, so its width and height would be the dimensions of the image, and the depth would be 3 (Red, Green, Blue channels).

CONVOLUTIONAL KERNEL



CONVOLUTIONAL KERNEL



Padding on the input volume with zeros in such way that the conv layer does not alter the spatial dimensions of the input

CONVOLUTION

These are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern
(3 x 3).

CONVOLUTION

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Dot
product

3

-1

CONVOLUTION

If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



CONVOLUTION

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

CONVOLUTION

stride=1

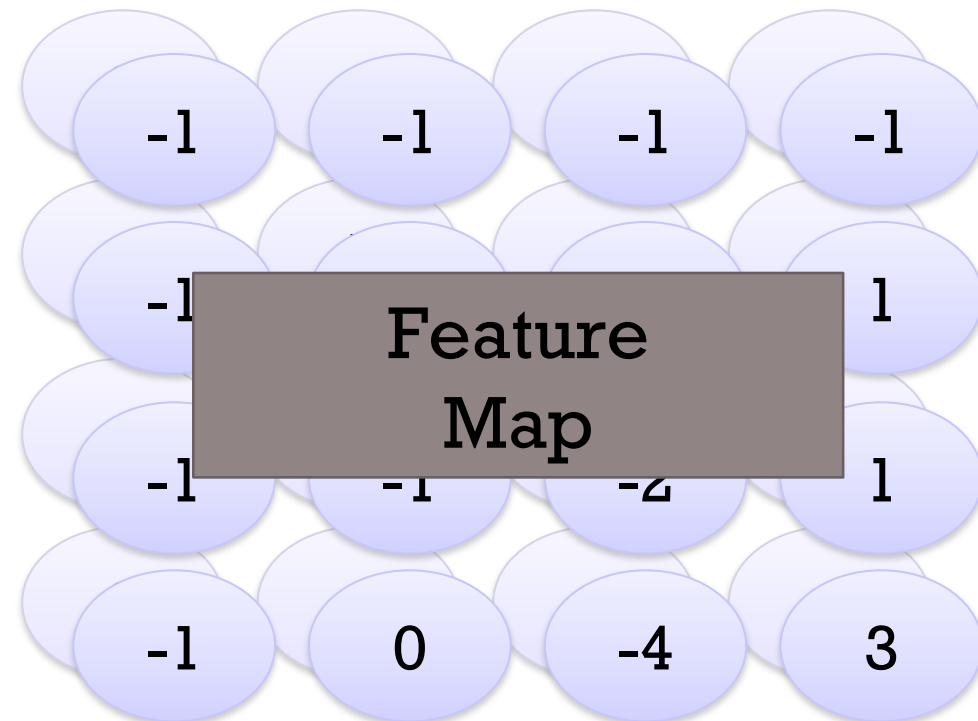
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

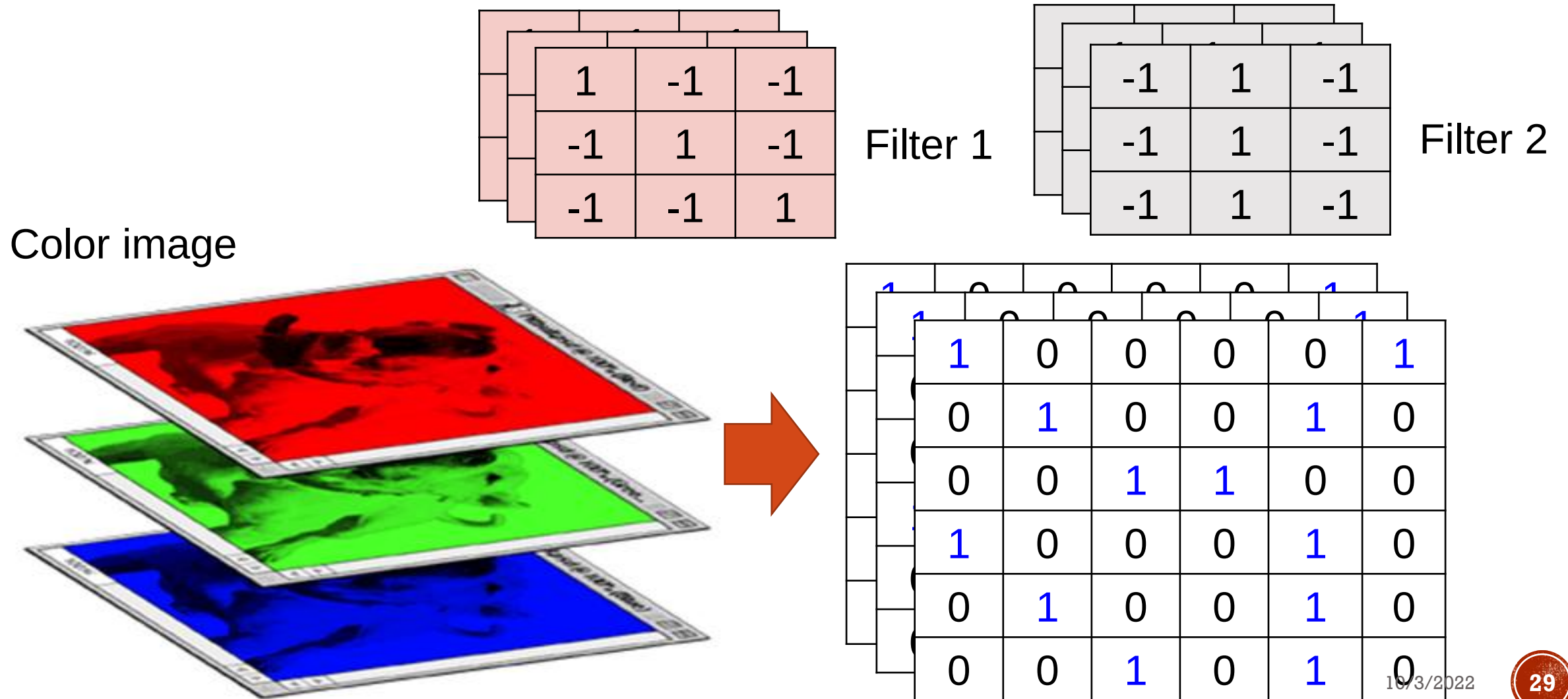
Repeat this for each filter



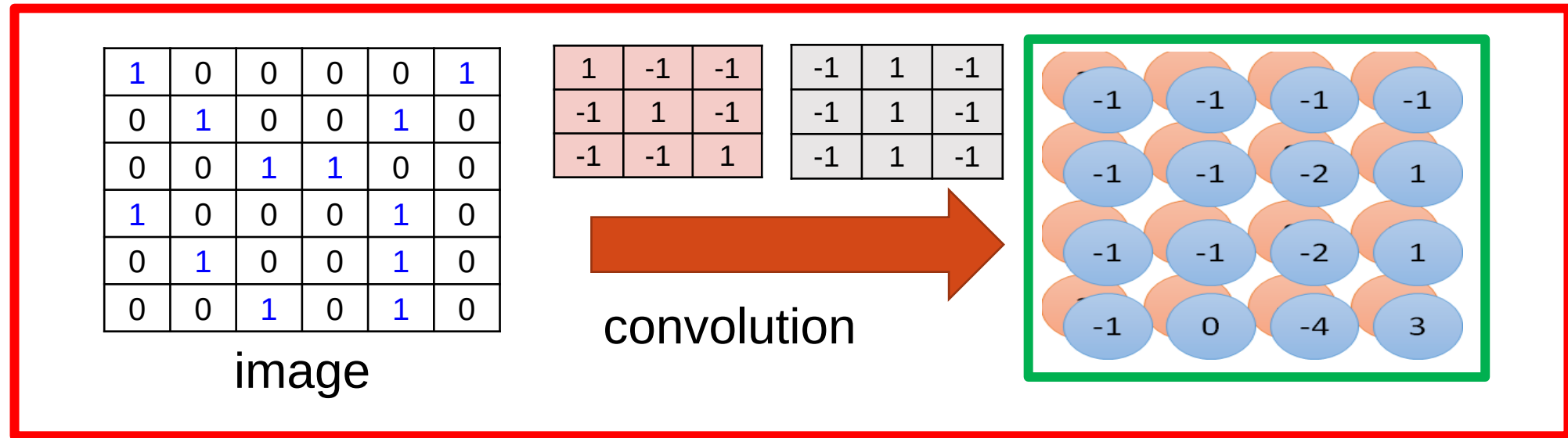
Two 4 x 4 images
Forming 2 x 4 x 4 matrix

10/3/2022

COLOR IMAGE: RGB 3 CHANNELS

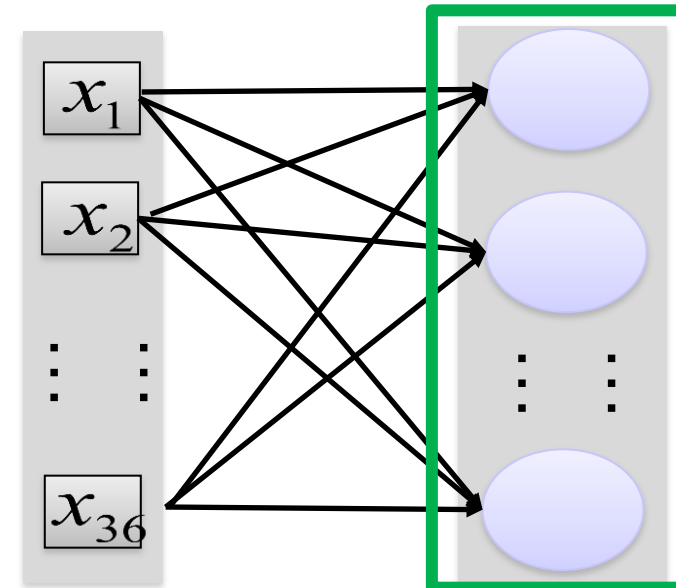


Convolution v.s. Fully Connected

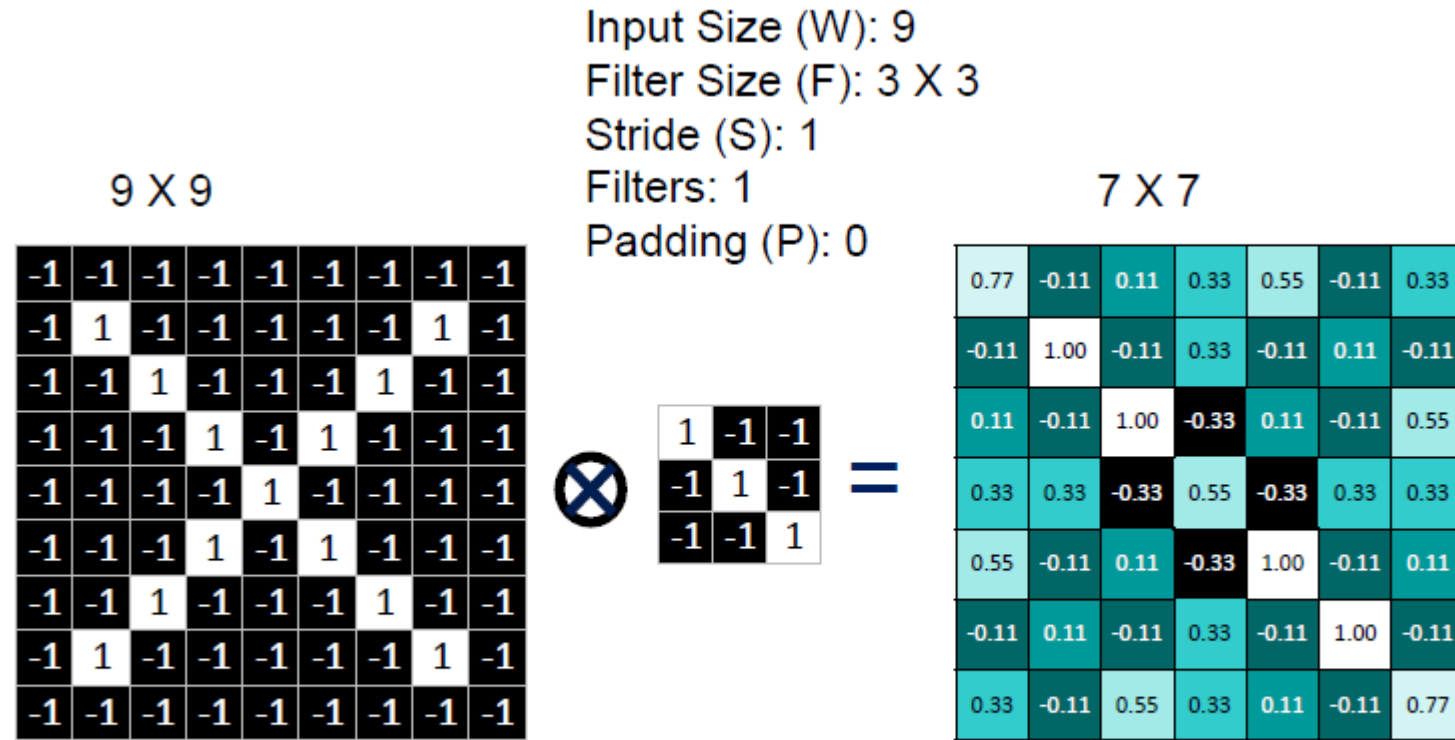


Fully-
connected

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



CONVOLUTIONAL LAYER – FILTERS – COMPUTATION EXAMPLE



$$\begin{aligned}\text{Feature Map Size} &= 1 + (W - F + 2P)/S \\ &= 1 + (9 - 3 + 2 \times 0)/1 = 7\end{aligned}$$

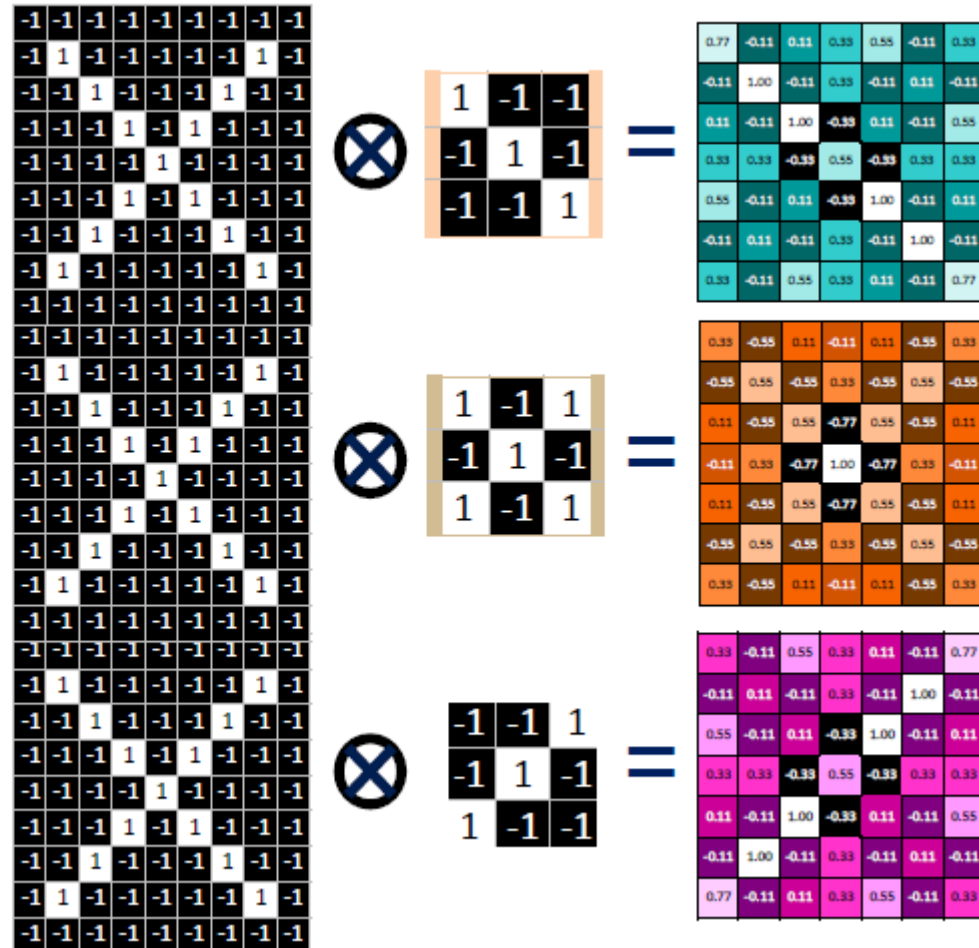
CONVOLUTIONAL LAYER – FILTERS – OUTPUT FEATURE MAP

- Output Feature Map of One complete convolution:

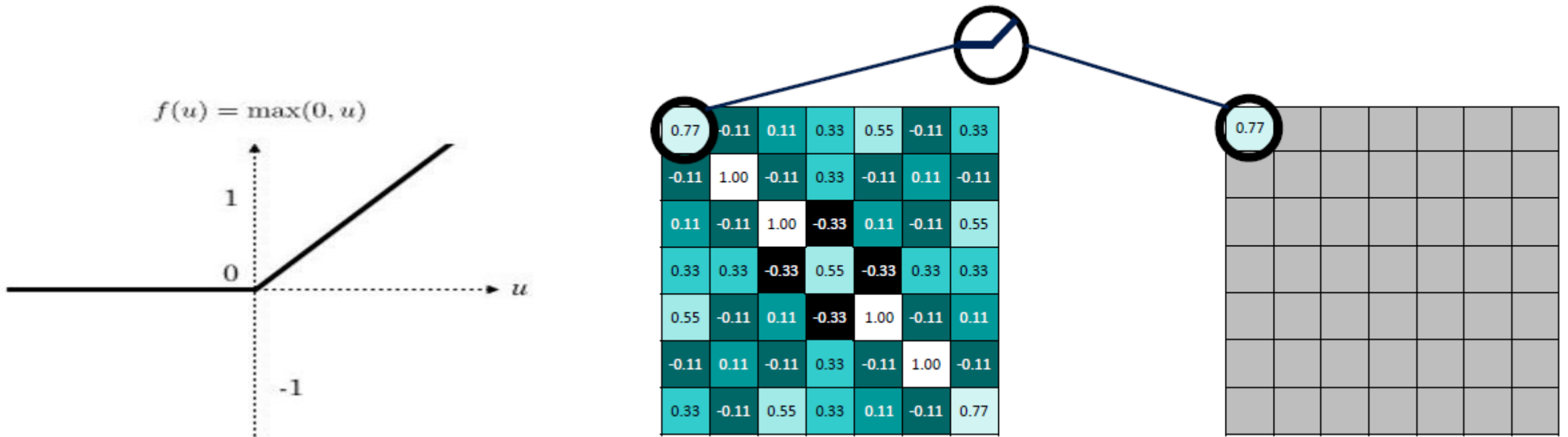
- Filters: 3
- Filter Size: 3 X 3
- Stride: 1

- Conclusion:

- Input Image:
9 X 9
- Output of Convolution:
7 X 7 X 3



RECTIFIED LINEAR UNIT (RELU)

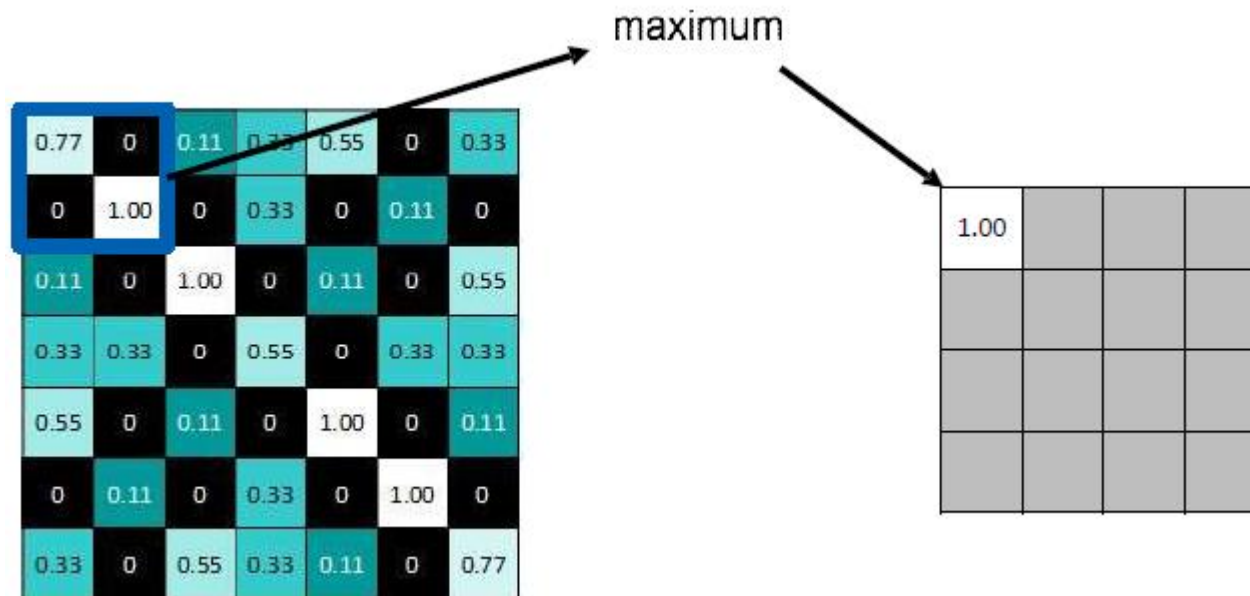


RECTIFIED LINEAR UNIT (RELU)

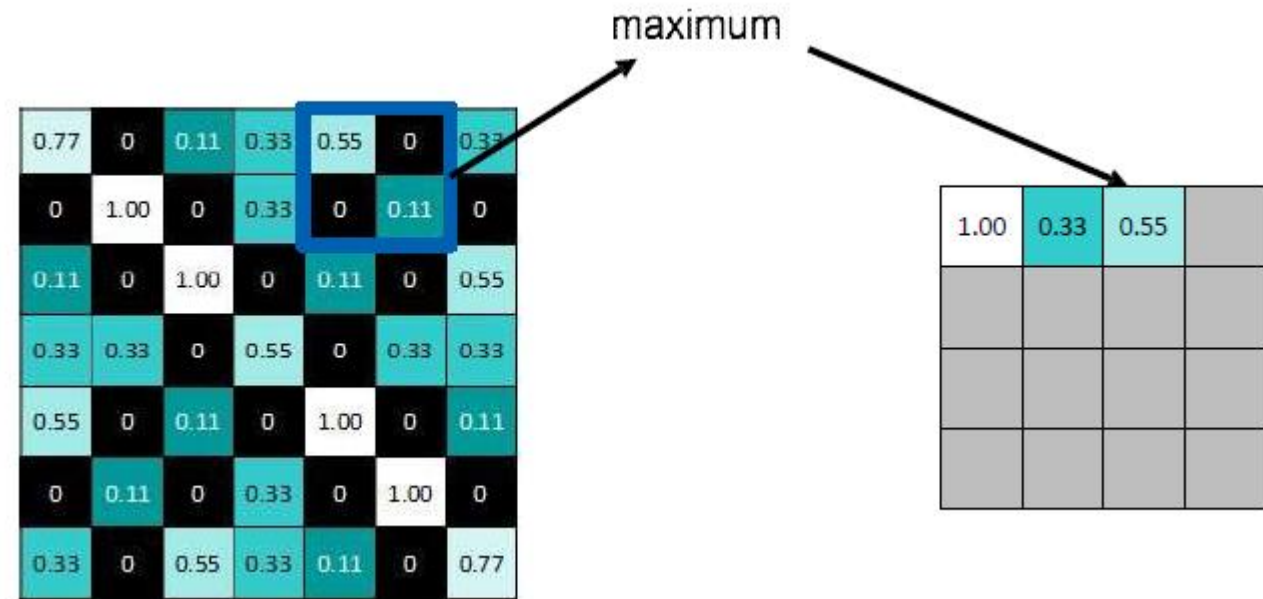
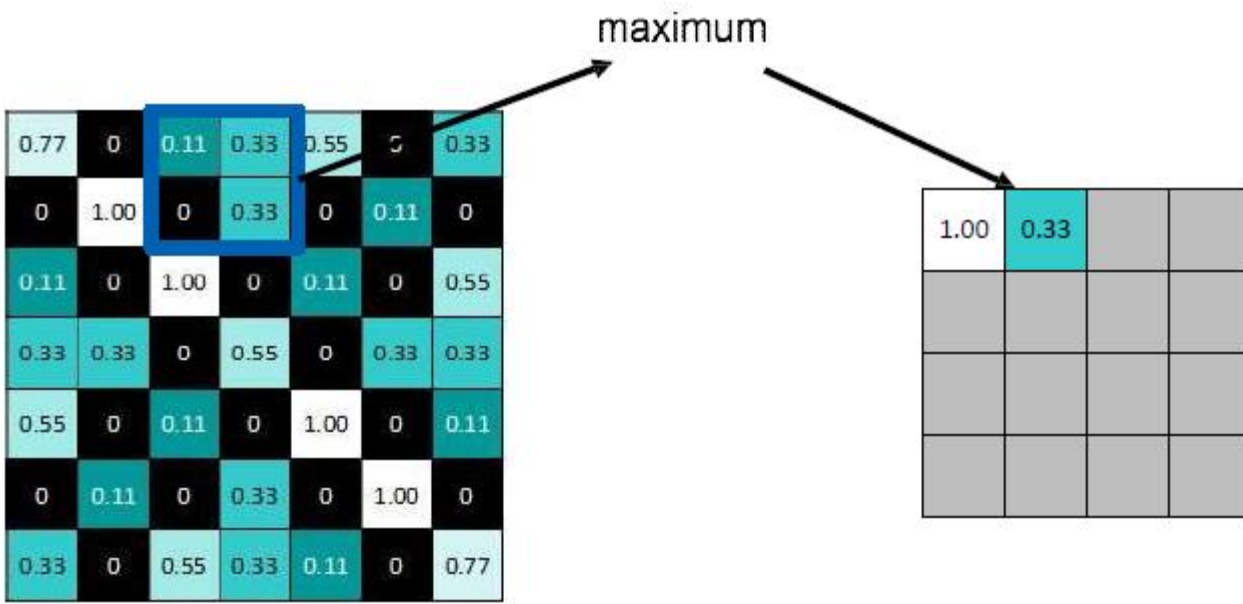


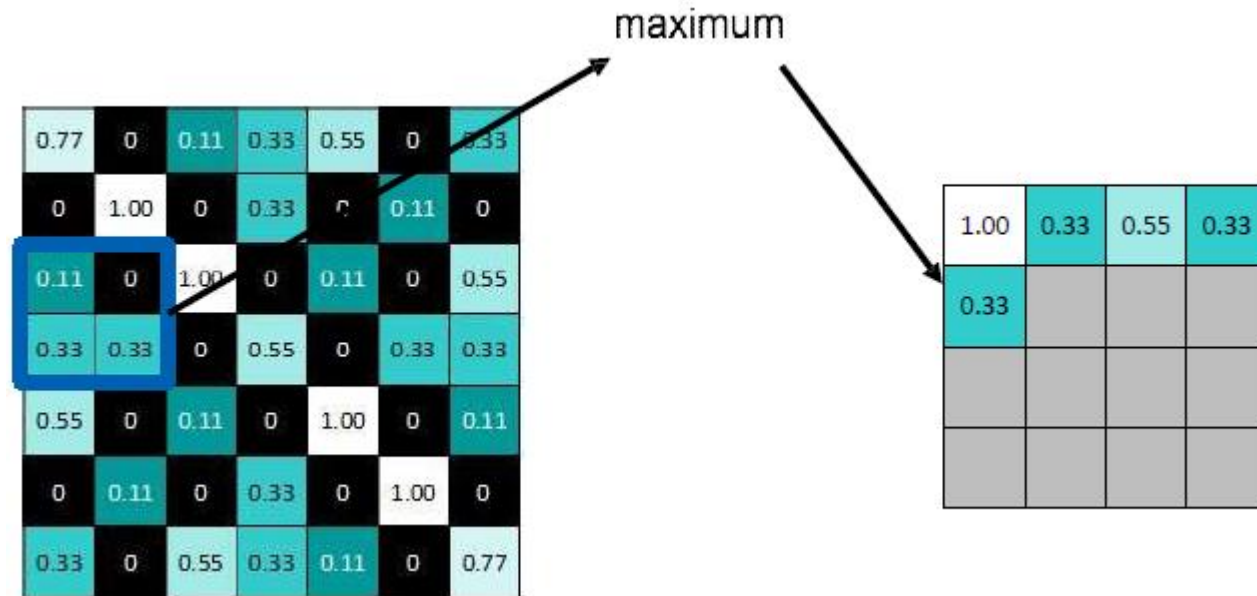
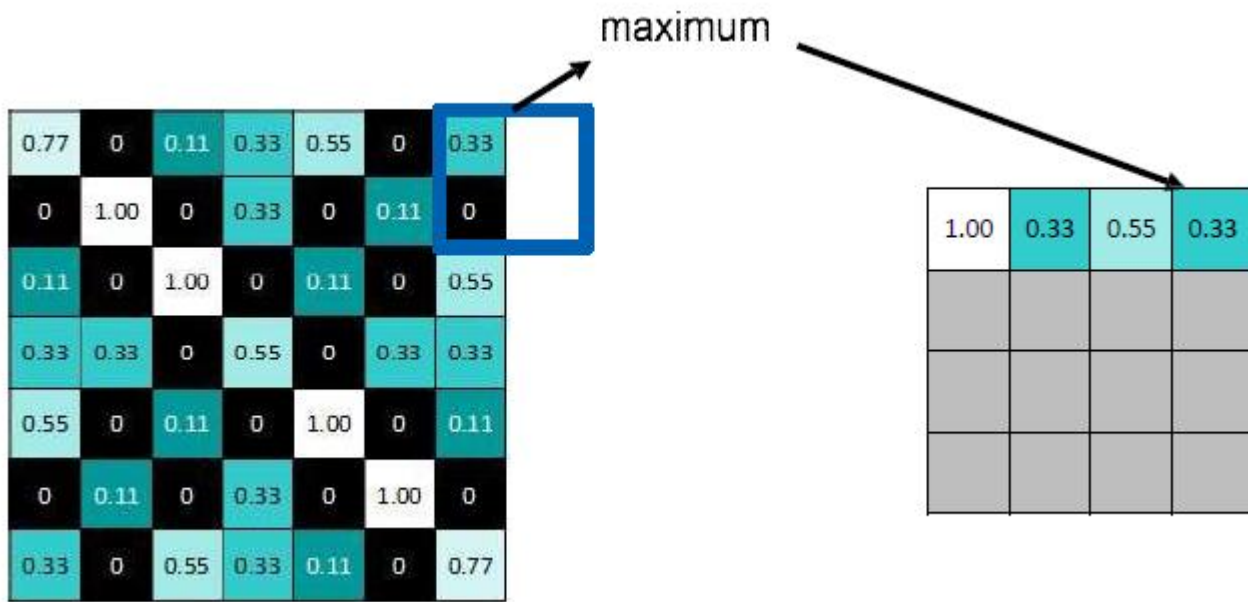
POOLING

- The pooling layers down-sample the previous layers feature map.
- Its function is to progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network
- The pooling layer often uses the Max operation to perform the downsampling process



Pooling Filter Size
= 2 X 2, Stride = 2





POOLING

0.77	0	0.11	0.33	0.55	0	0.33
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77

max pooling

1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

A stack of 3 larger images becomes a stack of smaller images.

0.77	0	0.11	0.33	0.55	0	0.11
0	1.00	0	0.33	0	0.11	0
0.11	0	1.00	0	0.11	0	0.55
0.33	0.33	0	0.55	0	0.33	0.33
0.55	0	0.11	0	1.00	0	0.11
0	0.11	0	0.33	0	1.00	0
0.33	0	0.55	0.33	0.11	0	0.77



1.00	0.33	0.55	0.33
0.33	1.00	0.33	0.55
0.55	0.33	1.00	0.11
0.33	0.55	0.11	0.77

0.33	0	0.11	0	0.11	0	0.55
0	0.55	0	0.33	0	0.55	0
0.11	0	0.55	0	0.55	0	0.11
0	0.11	0	1.00	0	0.11	0
0.33	0	0.55	0	0.55	0	0.11
0	0.33	0	0.33	0	0.33	0
0.33	0	0.11	0	0.11	0	0.33



0.55	0.33	0.55	0.33
0.33	1.00	0.55	0.11
0.55	0.55	0.55	0.11
0.33	0.11	0.11	0.33

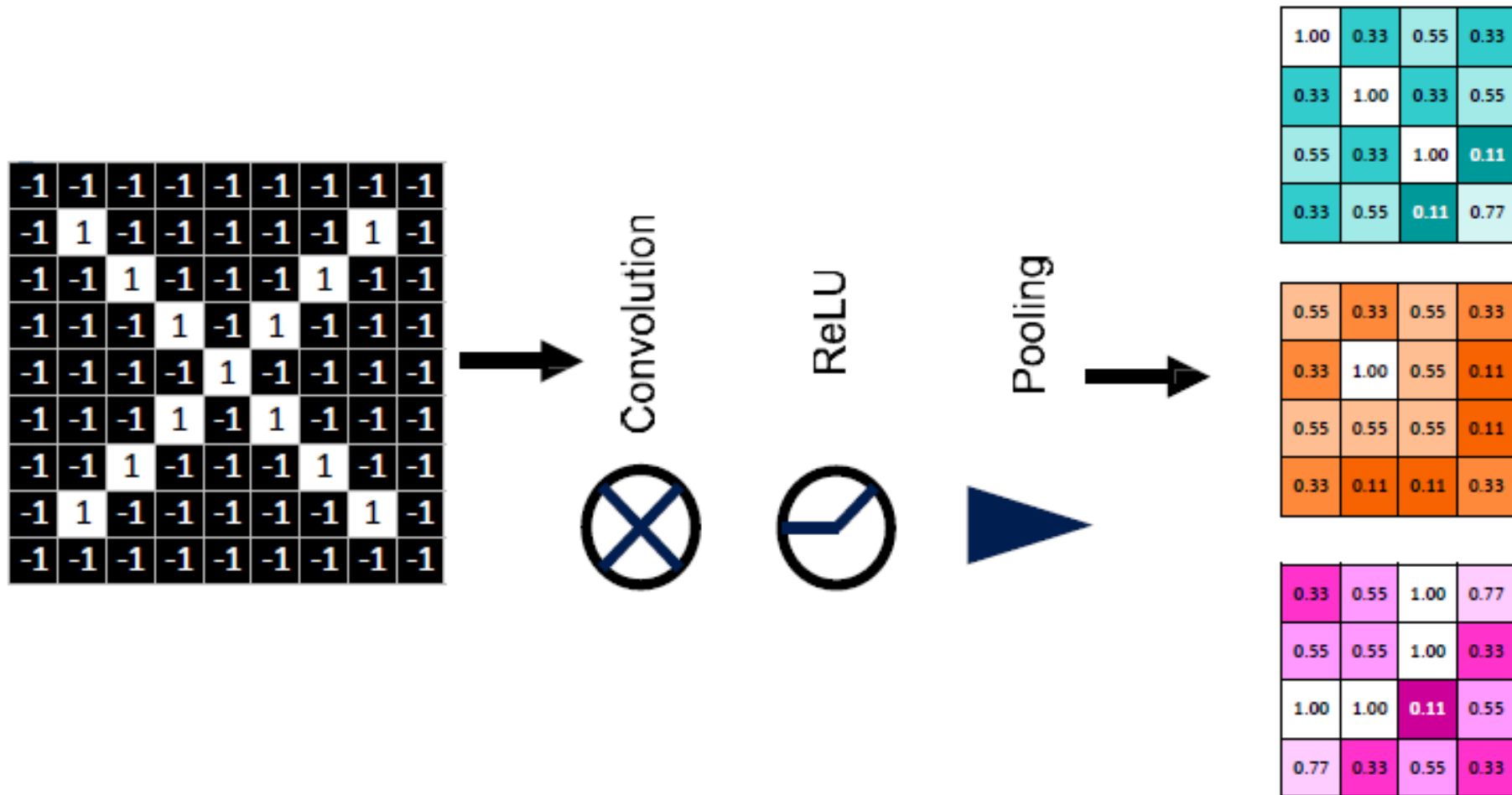
0.11	0	0.55	0.11	0.11	0	0.77
0	0.11	0	0.33	0	1.00	0
0.33	0	0.11	0	1.00	0	0.11
0.11	0.55	0	0.55	0	0.11	0.55
0.11	0	1.00	0	0.11	0	0.55
0	1.00	0	0.33	0	0.33	0
0.77	0	0.11	0.11	0.55	0	0.33



0.33	0.55	1.00	0.77
0.55	0.55	1.00	0.33
1.00	1.00	0.11	0.55
0.77	0.33	0.55	0.33

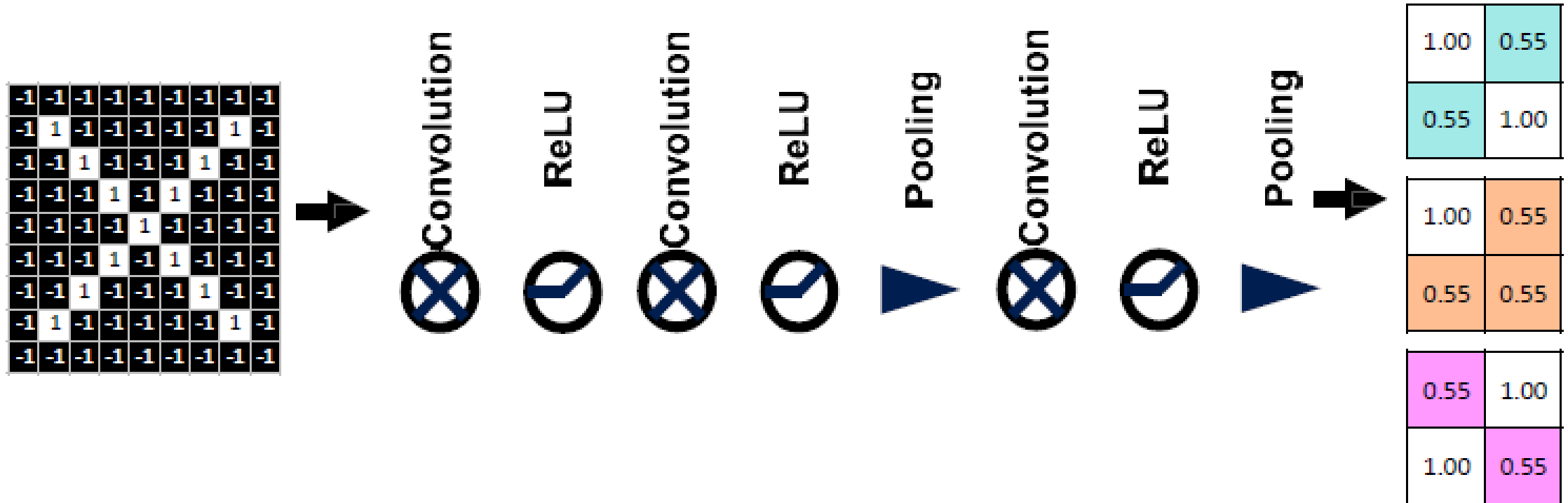
LAYERS GET STACKED

The output of one becomes the input of the next.



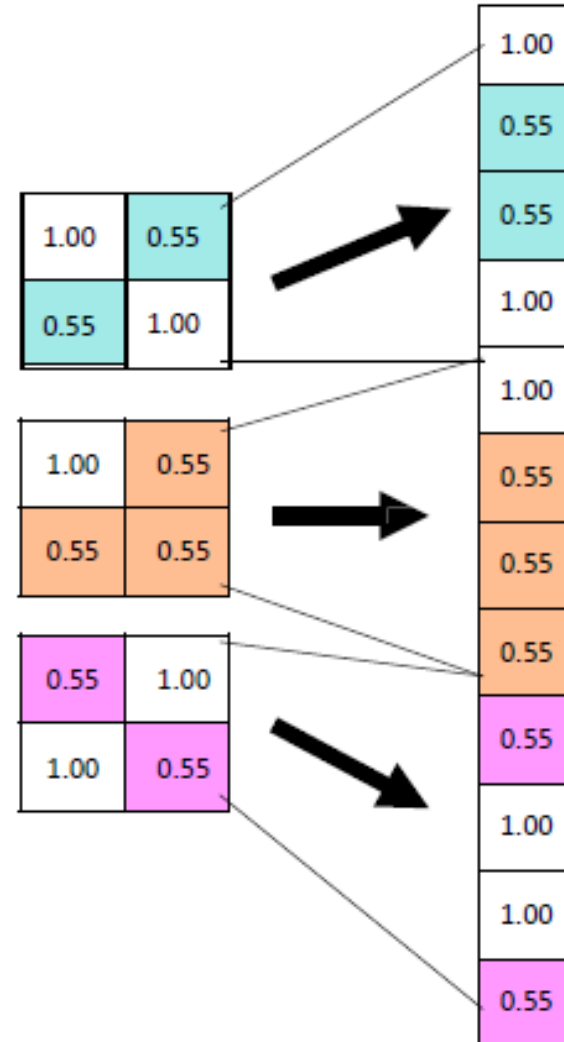
DEEP STAKING

Layers can be repeated several (or many) times.



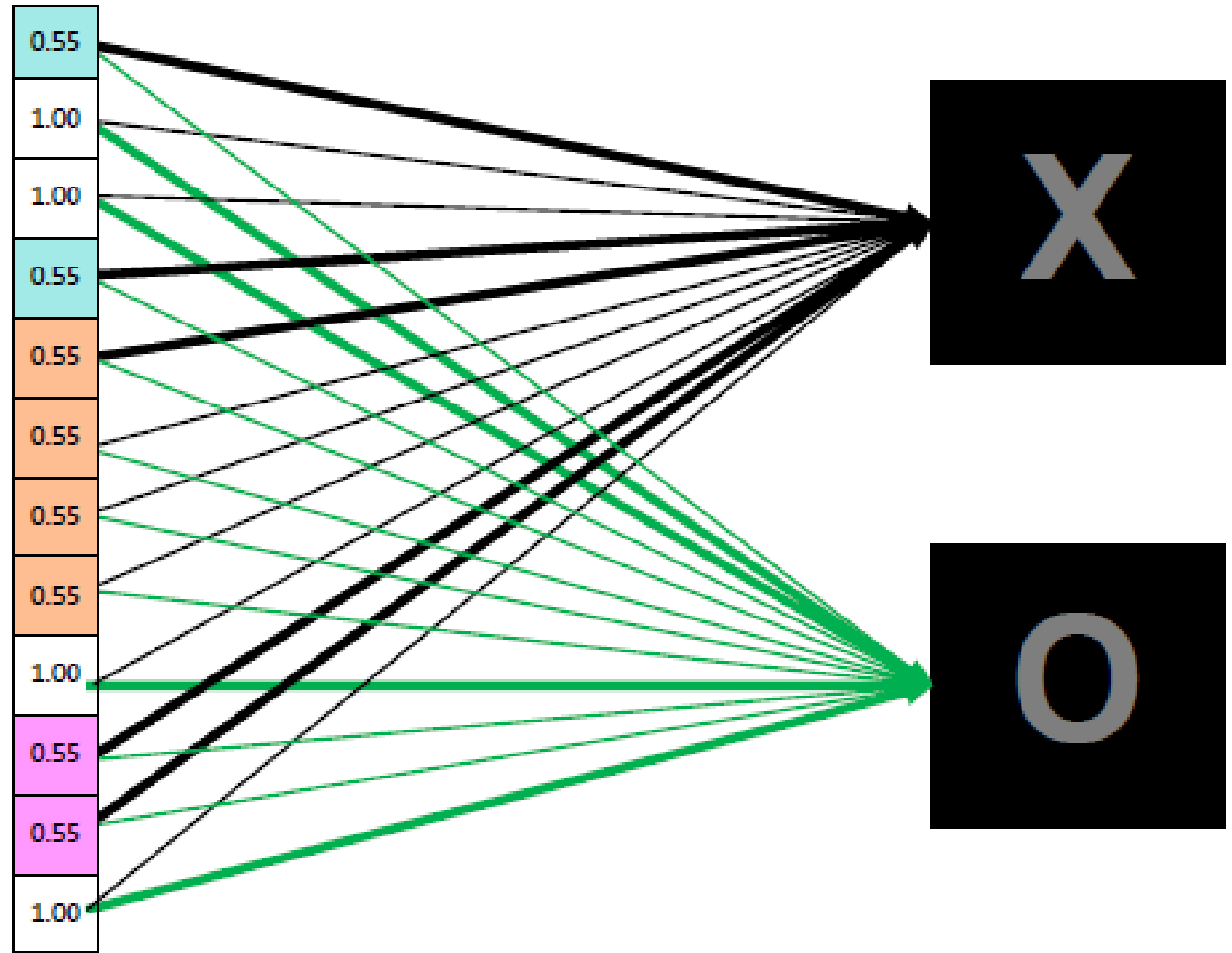
FULLY CONNECTED LAYER

- Fully connected layers are the normal flat feed-forward neural network layers.
- These layers may have a non-linear activation function or a softmax activation in order to predict classes.
- To compute our output, we simply rearrange the output matrices as a 1-D array.

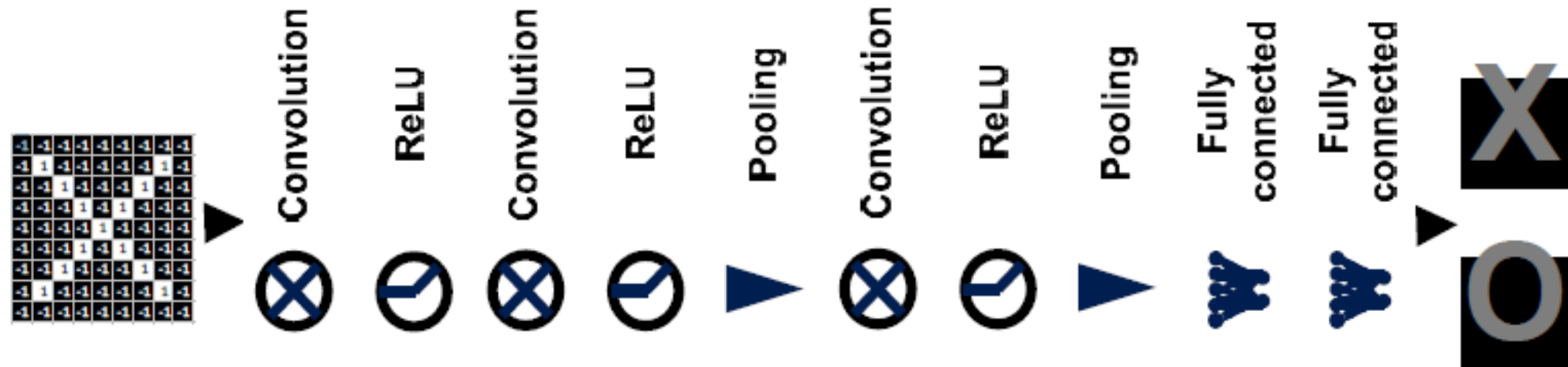


FC LAYER

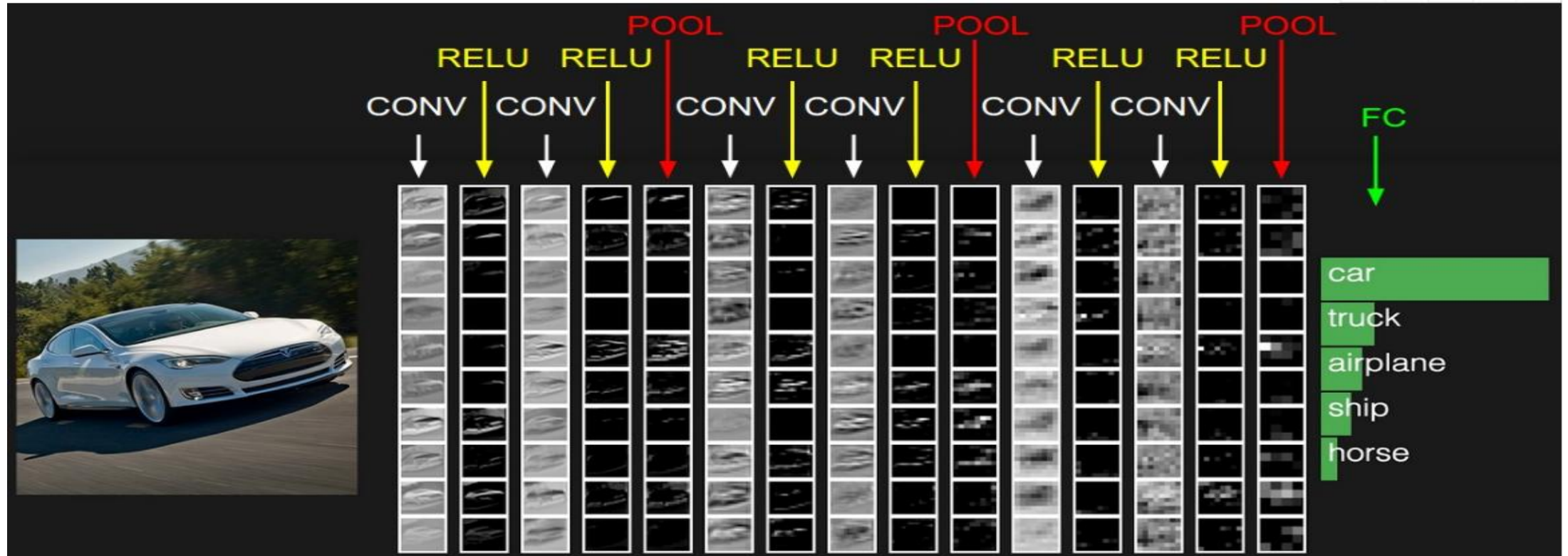
- A summation of product of inputs and weights at each output node determines the final prediction



PUTTING IT ALL TOGETHER

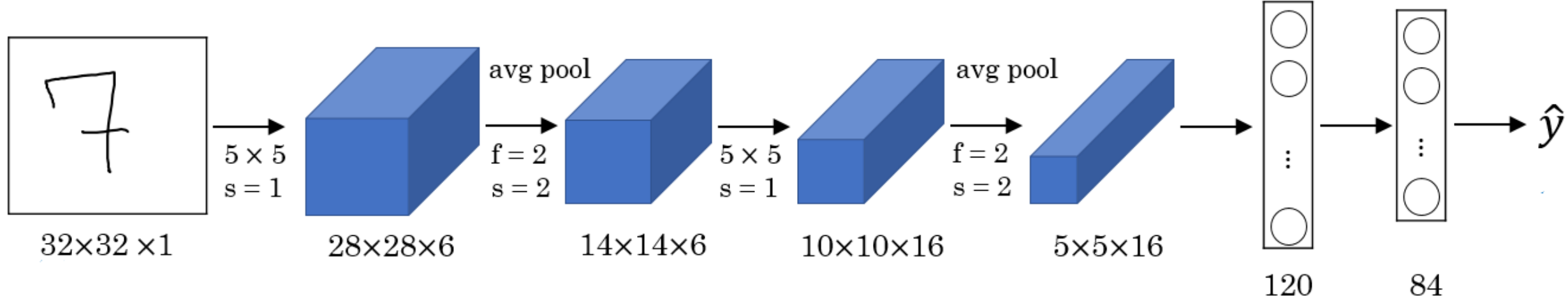


A SIMPLE CNN STRUCTURE



CONV: Convolutional kernel layer
RELU: Activation function
POOL: Dimension reduction layer
FC: Fully connection layer

LeNet



- The goal for this model was to identify handwritten digits in a 32x32x1 gray image.
- This model was published in 1998. The last layer wasn't using softmax back then.
- It has 60k parameters.
- The dimensions of the image decreases as the number of channels increases.
- Conv ==> Pool ==> Conv ==> Pool ==> FC ==> FC ==> softmax this type of arrangement is quite common.
- The activation function used in the paper was Sigmoid and Tanh. Modern implementation uses RELU in most of the cases.

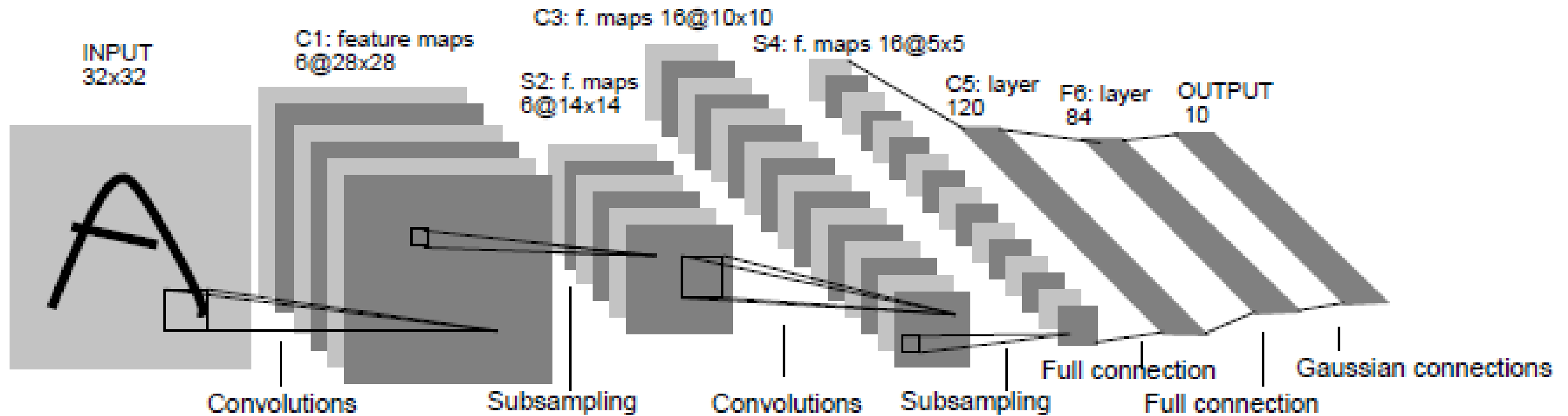


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

LeNet

Layer Type	Feature Map	Size	Kernel Size	Stride	Activation
Image	1	32×32	–	–	–
Convolution	6	28×28	5×5	1	tanh
Average Pooling	6	14×14	2×2	2	–
Convolution	16	10×10	5×5	1	tanh
Average Pooling	16	5×5	2×2	2	–
Fully Connected	–	120	–	–	tanh
Fully Connected	–	84	–	–	tanh
Fully Connected	–	10	–	–	softmax

LeNet

- “Gradient-Based Learning Applied To Document Recognition”.